

Proyecto Matera Inteligente

David Andrés Castellanos Ángulo

Julian Camilo Foreo Romero

Jeancarlo Arturo Rodriguez Ladino

Facultad de Ciencias de la Computación e Inteligencia Artificial

Universidad Sergio Arboleda

Arquitectura de Software

Juan Sebastián Leyva Casas

Septiembre 18 del 2026

Bogotá D.C

Diagrama de Paquetes

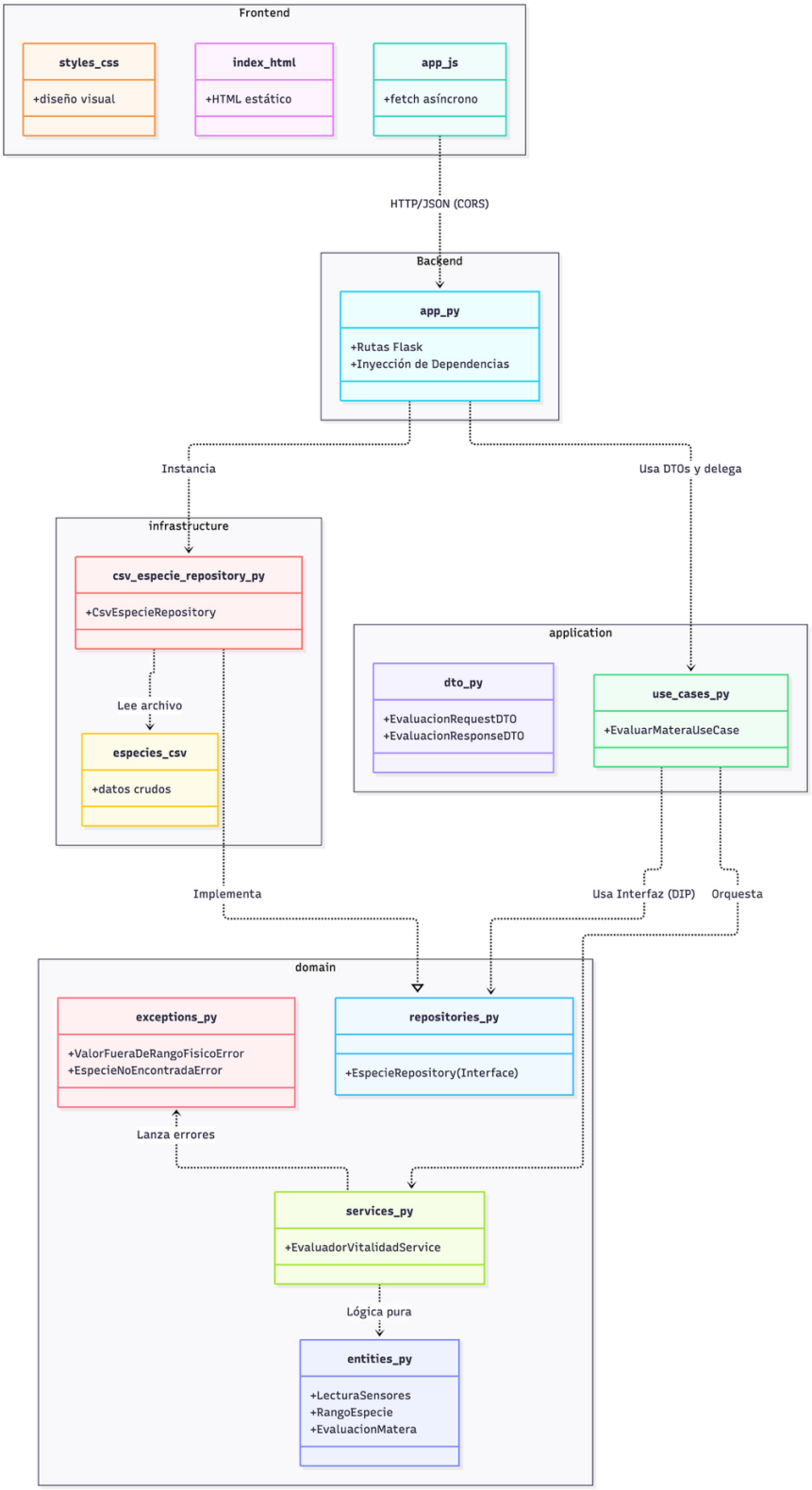


Diagrama de secuencia

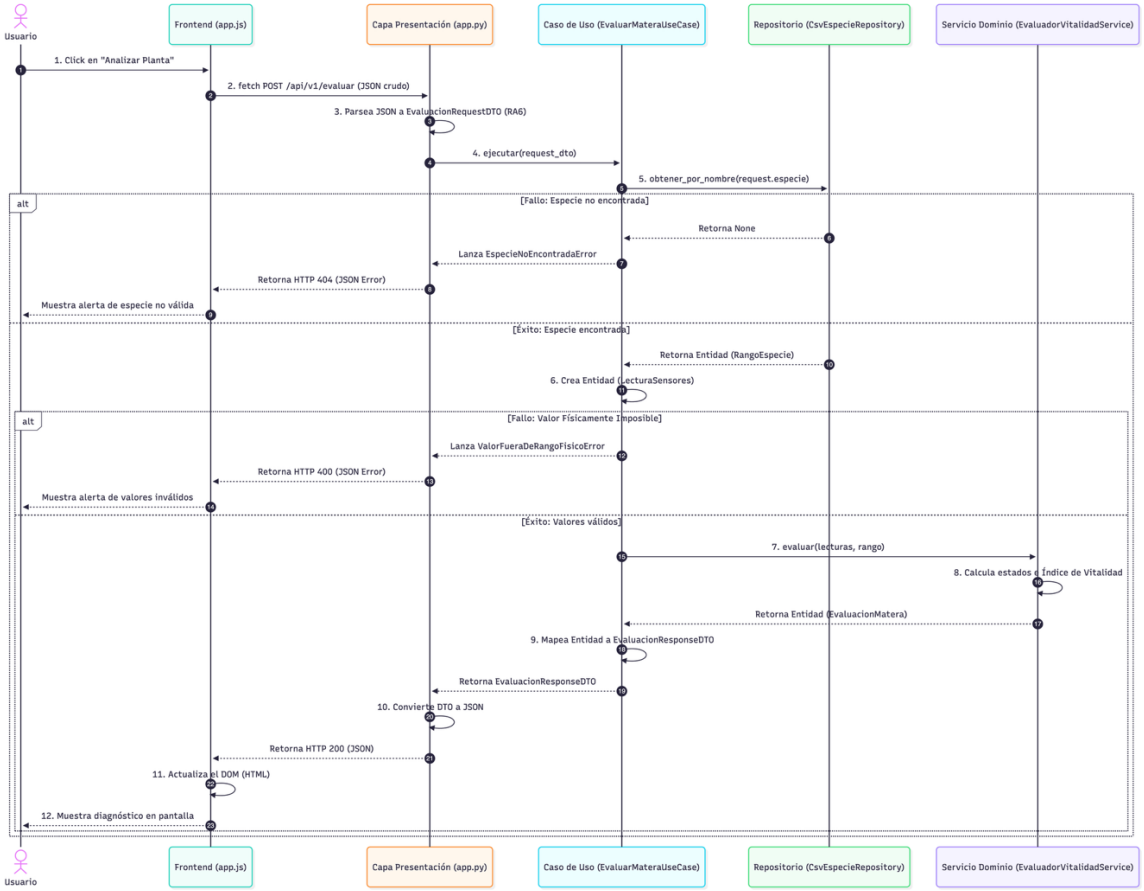


Tabla de Responsabilidades por Capa

Capa de Arquitectura	Responsabilidad	Que no debe hacer	Regla de la que depende
1. Domain	Contiene las entiedades del negocio como lecturaSensores, RangoEspecie y las reglas para evaluar la vitalidad de la planta (EvaluadorVitalidadService). Donde se definen las interfaces abstractas de los repositorios	No sabe la existencia de la web y tampoco lee archivos como la base de datos CSV o cualquier otra que se llegase a usar en un futuro.	No maneja dependiencia ya que que ninguna flecha de dependencia sale de esta capa

2. Application	Orquesta los casos de uso (EvaluarMateraUseCase) en donde coordina el flujo pidiendo datos al repositorio que luego se los pasa al dominio para que evalué, y formatea la respues en DTOs (EvaluaciónResponseDTO)	No contiene reglas de evalución del negocio como lo tiene la capa de Domain y tampoco importar librerias web com flask, tampoco debe saber de peticiones de red	Depende unicamente de la capa de dominio al usar sus entidades e interfaces abstractas
3. Infraestructure	Debe leer el archivo especies.csv y transforma las filas de texto en entidades de domino (CsvEspecieRepository)	No toma decisiones de neogcio o altera rangos, no define sus propios contratos si no que obedece a la interfaz abstracta que dicta el dominio	Depende del dominio para implementar sus interfaces abstractas y cosntruir sus entidades
4. Presentaición	Expone los endpoints restm en donde recibe los diccionarios JSON crudos de internet, los valida, los convierte en DTOs, inyecta las dependencias y responde con códigos HTTP	No evalua el estado de la planta, no lee el CSV, tampoco genera HTML renderizado del lado del servidor	Depende de la aplicación para mandar a ejecutar el caso de uso y de infraestructura para instanciarla

Justificación Prinpios Solid

S - Principio de Responsabilidad Única (SRP)

Dónde se aplicó: En el archivo Backend/domain/services.py, línea 10 (class EvaluadorVitalidadService). Esta clase tiene como única responsabilidad decidir el estado de la planta y calcular el índice de vitalidad. No es la clase que lee el CSV ni la que arma la respuesta HTTP en Flask.

O - Principio Abierto/Cerrado (OCP)

Dónde se aplicó: En el archivo Backend/domain/services.py, método _calcular_indice_vitalidad (línea 53). El servicio está abierto a la extensión pero cerrado a la modificación.

L - Principio de Sustitución de Liskov (LSP)

Dónde se aplicó: En el archivo Backend/infrastructure/csv_especie_repository.py, línea 16 (class CsvEspecieRepository(EspecieRepository)). Cualquier implementación concreta que creemos en el futuro (ej. PostgresEspecieRepository) podrá sustituir al CSV sin que el Dominio se rompa ni se entere del cambio.

I - Principio de Segregación de Interfaces (ISP)

Dónde se aplicó: En el archivo Backend/domain/repositories.py, línea 15 (class EspecieRepository(ABC)). La interfaz del Dominio depende estrictamente de las operaciones que necesita consultar ("dame los rangos de esta especie" y "dame todas las especies").

D - Principio de Inversión de Dependencias (DIP)

Dónde se aplicó: En el archivo Backend/application/use_cases.py, línea 19 (def __init__(self, especie_repository: EspecieRepository):). Es la formalización de la restricción RA5. El caso de uso (nivel superior) no depende del CSV (detalle técnico de nivel inferior), sino de la abstracción EspecieRepository.

Plan de Evolución

A continuación se detalla el impacto arquitectónico para los cuatro escenarios de evolución previstos, demostrando la alta cohesión y el bajo acoplamiento del sistema actual:

Escenario 1: Las mediciones dejan de llegar por HTTP y llegan por MQTT desde un ESP32

Componentes que se agregan: Un nuevo controlador en el borde del sistema (Capa de Presentación), por ejemplo, un archivo mqtt_listener.py que implemente un cliente Paho-MQTT suscrito a los topics del ESP32.

Componentes que se modifican: Exclusivamente la configuración de arranque en app.py para levantar el servicio MQTT en lugar de las rutas web de Flask.

Componentes que NO se tocan: Se garantiza que ninguna clase de la capa de Aplicación (use_cases.py, dto.py) ni de la capa de Dominio (entities.py, services.py) sufrirá modificaciones. El caso de uso seguirá recibiendo el mismo EvaluacionRequestDTO sin importarle si vino por red inalámbrica o cable.

Escenario 2: La tabla de referencia migra de CSV a una base de datos relacional

Componentes que se agregan: Una nueva clase en la capa de Infraestructura (ej. PostgresEspecieRepository) que implemente la interfaz abstracta EspecieRepository dictada por el dominio, usando SQL para abastecer los datos.

Componentes que se modifican: Única y exclusivamente la línea de Inyección de Dependencias en el borde del sistema (app.py), reemplazando la instanciación de CsvEspecieRepository por la de Postgres.

Componentes que NO se tocan: Gracias a la aplicación estricta del Principio de Inversión de Dependencias , el Dominio y el Caso de Uso no se tocan. El sistema central ignora por completo la existencia de la nueva base de datos.

Escenario 3: Aparecen usuarios, cada uno con varias plantas

Componentes que se agregan: Nuevas entidades en el Dominio (Usuario, MateraAsignada), nuevas interfaces de persistencia (UsuarioRepository), nuevos Casos de Uso (AutenticarUsuarioUseCase, RegistrarPlantaUseCase) y nuevos endpoints en la capa de Presentación con manejo de JWT/Sesiones.

Componentes que se modifican: La capa de Aplicación (EvaluarMateraUseCase) y los DTOs, ya que ahora el caso de uso deberá validar a qué usuario le pertenece la planta antes de evaluarla.

Componentes que NO se tocan: El servicio central matemático (EvaluadorVitalidadService) y las entidades botánicas (RangoEspecie). La regla biológica universal de que una Orquídea necesita 50% de humedad permanece intacta y no cambia por el hecho de que la planta ahora tenga un dueño.

Escenario 4: Se agrega gamificación (puntos, rachas, niveles) sobre el cuidado de la planta

Componentes que se agregan: Un nuevo sub-dominio o paquete enfocado netamente en gamificación, con sus propias entidades (Logro, Puntaje, Racha), repositorios y servicios de dominio aislados.

Componentes que se modifican: La capa de Aplicación. El orquestador (EvaluarMateraUseCase) deberá disparar un evento (Event-Driven) o llamar al nuevo servicio de gamificación cuando detecte que la evaluación resultó en un índice de vitalida excelente, para así otorgar los puntos.

Componentes que NO se tocan: La lógica biológica existente (EvaluadorVitalidadService). Sería una violación al Principio de Responsabilidad Única (SRP) obligar al evaluador a sumar puntos La biología y la gamificación se mantienen totalmente desacopladas

Decisiones y Alternativas descartadas

1. Decisión de diseño: Usar un Patrón DTO (Data Transfer Object) en el borde del sistema

Alternativa considerada: Devolver directamente la entidad de dominio pura (EvaluacionMatera) desde la capa de aplicación hacia el controlador de Flask, forzando su serialización a JSON allí mismo.

Razón del descarte: Hacerlo habría acoplado fuertemente la estructura interna del negocio con el contrato de la API web. Si en el futuro el Frontend requiere que los datos se llamen distinto, habríamos tenido que modificar la entidad biológica. Al usar EvaluacionResponseDTO, garantizamos que el Dominio dicte las reglas internamente, mientras la capa de Aplicación moldea la forma de los datos para el exterior, respetando el SRP.

2. Decisión de diseño: Mapear las excepciones a códigos HTTP exclusivamente en la Capa de Presentación

Alternativa considerada: Programar dentro de las Entidades o Servicios del Dominio sentencias de framework web, como abort(400) o devolver un diccionario con {"status": 404}, cuando se ingresaba una especie desconocida o valores físicos negativos.

Razón del descarte: Esta alternativa fue rechazada porque violaba la restricción arquitectónica RA4. El Dominio debe ser completamente agnóstico al protocolo de red. El Dominio se limitó a lanzar excepciones puras de Python (EspecieNoEncontradaError). Fue el controlador (app.py) quien asumió la responsabilidad de atrapar ese error y decidir que, en el contexto de la web, eso equivale a un código HTTP 404.

3. Decisión de diseño: Inyectar el repositorio de CSV desde el punto de arranque del framework

Alternativa considerada: Instanciar la clase concreta CsvEspecieRepository() directamente dentro del constructor de EvaluarMateraUseCase o dentro del servicio de Dominio.

Razón del descarte: Se descartó porque creaba un alto acoplamiento hacia un detalle técnico de infraestructura, violando el Principio de Inversión de Dependencias y la restricción RA5. Si hubiésemos tomado esa alternativa, el día que migremos a PostgreSQL o MySQL tendríamos que modificar archivos internos de la capa de Aplicación. Al instanciar el CSV en app.py y pasarlo por parámetro, el núcleo se mantiene ciego y protegido frente al mecanismo de abastecimiento de datos.

Prompts Usados

Actúa como un ingeniero en ciencias de la computación e IA. Ayúdame con el paso a paso sin hacerme el siguiente ejercicio, la idea es aprender y hacer las cosas por sí solo. El ejercicio es de mi materia que se llama Arquitectura de software, ya tengo creado el documento word para empezar a agregar las actividades al informe y ya cree el git hub con las siguientes ramas features, staging , develop y main

Actuando como ingeniero en ciencias de la computación e IA, enseñame sin hacer las cosas tu, es decir debes instruirme como tun aprendiz junior a crear el endpoint con flask

ahora debo finalizar con esto Mínimo seis pruebas que ejerciten las reglas de negocio sin levantar el servidor web y sin leer el archivo o la base dedatos reales (usando dobles de prueba de la abstracción de RA5). Estas pruebas son la evidencia ejecutable de RA4.